

Recommended 3-Day Mobile Stack

Use this stack for fastest execution

Mobile App: React Native + Expo

Language: TypeScript

Styling: NativeWind / React Native StyleSheet

Backend: Supabase Edge Functions OR Node/Express API

Database: Supabase PostgreSQL

Vector Search: For hackathon, start with local JSON keyword retrieval

AI: OpenAI / Gemini API

Deployment: Expo Go demo + EAS build later

Best hackathon choice

For 3 days:

React Native Expo + TypeScript

Local JSON RAG + AI API

Why?

- Fastest to build
- Easy to demo on phone
- No Play Store needed
- Can run in Expo Go
- Backend can be minimal

Overall Phases

Phase 0: Product Scope Lock

Phase 1: Project Setup

Phase 2: UI Screens

Phase 3: Bhagavad Gita Knowledge Base

Phase 4: Emotion + Risk Detection

Phase 5: RAG Retrieval Engine

Phase 6: AI Response Engine

Phase 7: Core Chat Flow

Phase 8: Admin Content Generator
Phase 9: Query Limit + Subscription Mock
Phase 10: Testing + Demo Scenarios
Phase 11: Deployment + Final Demo

Phase 0: Product Scope Lock

Goal

Freeze exactly what you will build in 3 days.

Build only these modules

User App

- Onboarding screen
- Category selection
- Chat screen
- AI response with:
 - Emotion label
 - Risk level
 - PausePulse prompt
 - Bhagavad Gita verse
 - Reflective answer
 - ForwardMirror outcome
- Crisis response screen
- Free query counter

Admin Mode

- Admin content generator
- Generate:
 - Reel script
 - Blog outline
 - Instagram caption
 - Image prompt

Do not build now

- Real payment

- Real login
- Therapist dashboard
- Real-time voice calls
- Multi-religion support
- Wearable integration
- Production-grade analytics

Cursor Prompt

We are building a 3-day hackathon MVP mobile app called AatmaMirror + DharmaMind AI.

Scope:

- React Native Expo app
- User can select emotional category
- User can chat with AI
- App detects emotion/risk
- App retrieves Bhagavad Gita verse from local JSON
- App generates reflective AI answer
- App handles crisis messages safely
- Admin can generate wellness content

Do not build auth, payments, or complex backend yet.
Create clean modular code with TypeScript.

Phase 1: Project Setup

Goal

Create the mobile app foundation.

Tasks

1. Create Expo app

```
npx create-expo-app aatma-mirror --template
```

Choose:

Blank TypeScript

2. Install dependencies

```
cd aatma-mirror
```

```
npm install @react-navigation/native
npm install @react-navigation/native-stack
npm install react-native-screens react-native-safe-area-context
npm install react-native-gesture-handler
npm install expo-linear-gradient
npm install lucide-react-native
npm install zustand
npm install axios
npm install react-native-dotenv
```

Optional styling:

```
npm install nativewind tailwindcss
```

3. Create folder structure

```
src/
├── app/
├── screens/
│   ├── OnboardingScreen.tsx
│   ├── CategoryScreen.tsx
│   ├── ChatScreen.tsx
│   ├── AdminScreen.tsx
│   └── DemoScreen.tsx
├── components/
│   ├── ChatBubble.tsx
│   ├── EmotionBadge.tsx
│   ├── RiskBadge.tsx
│   ├── VerseCard.tsx
│   ├── PausePulseCard.tsx
│   ├── ForwardMirrorCard.tsx
│   ├── CrisisAlert.tsx
│   ├── CategoryCard.tsx
│   └── LoadingReflection.tsx
├── data/
│   └── gitaVerses.ts
├── services/
│   ├── aiService.ts
│   ├── ragService.ts
│   ├── safetyService.ts
│   └── contentService.ts
├── store/
└── useAppStore.ts
```

```
|— types/
|   |— index.ts
|— constants/
|   |— categories.ts
|   |— constitution.ts
```

Cursor Prompt

Create a React Native Expo TypeScript project structure for a mobile app named AatmaMirror.

Create folders:

src/screens, src/components, src/services, src/data, src/store, src/types, src/constants.

Set up React Navigation with these screens:

- OnboardingScreen
- CategoryScreen
- ChatScreen
- AdminScreen
- DemoScreen

Use clean TypeScript types and placeholder UI.

Output of Phase 1

- App runs in Expo Go
 - Navigation works
 - Empty screens available
-

Phase 2: UI Screens

Goal

Build beautiful mobile UI for demo.

Screen 1: OnboardingScreen

Content

- App name: **AatmaMirror**

- Subtitle: **Reflect before you react**
- Description: **AI-guided mental wellness inspired by Bhagavad Gita wisdom**
- Button: **Start Reflection**
- Small disclaimer: **Not a replacement for professional therapy**

Screen 2: CategoryScreen

Categories

- Exam Stress
- Career Confusion
- Relationship Conflict
- Anger Control
- Loneliness
- Motivation
- Fear of Failure

Each card should have:

- Icon
- Title
- Short subtitle

Screen 3: ChatScreen

Sections

- Header with selected category
- Query counter: **Free queries left: 5**
- Chat messages
- Text input
- Send button
- AI response cards:
 - EmotionBadge
 - RiskBadge
 - PausePulseCard
 - VerseCard
 - ForwardMirrorCard

Screen 4: AdminScreen

Content

- Topic input
- Content type selector:
 - Reel
 - Blog
 - Caption
 - Image Prompt
- Generate button
- Output card

Screen 5: DemoScreen

Content

Pre-filled demo buttons:

- Exam stress demo
- Anger demo
- Crisis demo
- Admin content demo

Cursor Prompt

Build polished React Native screens for the AatmaMirror app.

Design style:

- Dark purple/blue gradient background
- Clean cards
- Rounded corners
- Soft shadows
- Student wellness theme
- Use lucide-react-native icons

Screens:

1. OnboardingScreen
2. CategoryScreen
3. ChatScreen
4. AdminScreen
5. DemoScreen

Keep UI demo-ready and mobile responsive.

Output of Phase 2

- Demo-ready mobile UI
 - Navigation between screens
 - No AI yet
-

Phase 3: Bhagavad Gita Knowledge Base

Goal

Create local scripture dataset for RAG.

Dataset Size for Hackathon

Use 25–40 verses.

Categories to cover

stress
exam
career
anger
desire
fear
confusion
discipline
detachment
relationship
loneliness
motivation

Data Format

Create:

```
src/data/gitaVerses.ts
export const gitaVerses = [
  {
    id: "gita-2-47",
    chapter: 2,
    verse: 47,
```



```

    sanskrit: "",
    translation: "You have a right to perform your duty, but not to the fruits of action.",
    meaning: "Focus on sincere effort instead of being consumed by results.",
    tags: ["stress", "exam", "career", "detachment", "duty"],
    categories: ["Exam Stress", "Career Confusion", "Motivation"]
  },
  {
    id: "gita-2-63",
    chapter: 2,
    verse: 63,
    translation: "From anger comes delusion; from delusion, loss of memory; from loss of memory,
destruction of intelligence.",
    meaning: "Anger clouds judgment and leads to harmful decisions.",
    tags: ["anger", "conflict", "impulse", "relationship"],
    categories: ["Anger Control", "Relationship Conflict"]
  }
];

```

Recommended MVP Verses

Add verses around:

- 2.47 — duty and results
- 2.48 — equanimity
- 2.50 — skill in action
- 2.56 — steady mind
- 2.62–63 — desire and anger
- 3.19 — detached action
- 4.39 — faith and knowledge
- 6.5 — uplift yourself
- 6.6 — mind as friend/enemy
- 6.26 — bring mind back
- 12.13–14 — compassion and balance
- 18.66 — surrender and trust

Cursor Prompt

Create a TypeScript file `src/data/gitaVerses.ts`.

Add 30 Bhagavad Gita verse objects for mental wellness use cases:

- exam stress
- career confusion
- anger
- relationship conflict

- fear
- loneliness
- motivation
- self-control
- detachment
- discipline

Each object should include:

id, chapter, verse, translation, meaning, tags, categories.

Keep translations concise and suitable for demo.

Output of Phase 3

- Local Gita dataset ready
 - Can retrieve verses by tags/categories
-

Phase 4: Emotion + Risk Detection

Goal

Detect user emotion and crisis risk.

File

src/services/safetyService.ts

Functions

```
detectEmotion(message: string): EmotionType
detectRiskLevel(message: string): RiskLevel
detectCategory(message: string): string
isCrisisMessage(message: string): boolean
```

Types

```
export type EmotionType =
  | "stress"
  | "sadness"
```

| "anger"
| "fear"
| "confusion"
| "loneliness"
| "motivation"
| "neutral";

export type RiskLevel = "LOW" | "MEDIUM" | "HIGH" | "CRISIS";

Rule-Based Keywords

Stress

stress, pressure, exam, deadline, overwhelmed, tension

Anger

angry, hate, rage, irritated, insult, fight

Sadness

sad, crying, lonely, broken, depressed

Fear

scared, afraid, anxious, worried, panic

Crisis

suicide, kill myself, end my life, don't want to live, self harm, hurt myself

Crisis Response

If crisis detected, return:

I'm really sorry you're feeling this way. This sounds urgent and you should not face it alone.
Please contact someone you trust immediately or reach out to emergency support in your area.
If you are in immediate danger, call local emergency services now.

Cursor Prompt

Create src/services/safetyService.ts.

Implement:

- detectEmotion(message)
- detectRiskLevel(message)

- isCrisisMessage(message)
- getCrisisResponse()

Use rule-based keyword matching for hackathon MVP.
Return TypeScript-safe emotion and risk labels.

Output of Phase 4

- Emotion detection works
 - Crisis messages are safely handled
 - Risk badge can be shown in UI
-

Phase 5: RAG Retrieval Engine

Goal

Retrieve relevant Bhagavad Gita verses.

File

src/services/ragService.ts

MVP Approach

Use keyword + tag scoring.

No vector DB needed for 3-day demo.

Algorithm

1. Lowercase user message
2. Extract words
3. Score each verse:
 - +3 if category matches selected category
 - +2 if tag appears in message
 - +1 if keyword appears in meaning
4. Sort by score
5. Return top 3 verses

Function

```
retrieveRelevantVerses({  
  message,  
  category,  
  emotion  
}): GitaVerse[]
```

Cursor Prompt

Create src/services/ragService.ts.

Implement a lightweight RAG retrieval function using local gitaVerses data.

Function:

```
retrieveRelevantVerses(message: string, category: string, emotion: string): GitaVerse[]
```

Scoring:

- category match: +3
- tag match in message: +2
- emotion match in tags: +2
- keyword match in meaning: +1

Return top 3 verses sorted by score.

Output of Phase 5

- User query returns relevant Gita verses
 - Works offline/local
 - Fast enough for demo
-

Phase 6: AI Response Engine

Goal

Generate final reflective response.

File

src/services/aiService.ts

Option A: Use Real LLM API

Use OpenAI/Gemini.

Option B: Fallback Template

If API fails, use template-based response.

Response Shape

```
export type ReflectiveAIResponse = {  
  emotion: EmotionType;  
  riskLevel: RiskLevel;  
  pausePrompt: string;  
  answer: string;  
  verses: GitaVerse[];  
  forwardMirror: {  
    impulsivePath: string;  
    reflectivePath: string;  
    suggestedAction: string;  
  };  
};
```

Prompt Template

You are AatmaMirror, a reflective AI mental wellness guide.

Rules:

- Do not blindly validate the user.
- Do not give medical diagnosis.
- Do not claim to be Krishna, God, guru, or therapist.
- Use only the provided Bhagavad Gita context.
- Encourage pause, reflection, and wise action.
- Keep answer short and practical.
- Always include a reflection question.
- Mention that the guidance is inspired by Bhagavad Gita teachings.

User problem:

{message}

Emotion:

{emotion}

Category:

{category}

Relevant Bhagavad Gita context:

{verses}

Generate JSON:

```
{
  "pausePrompt": "",
  "answer": "",
  "forwardMirror": {
    "impulsivePath": "",
    "reflectivePath": "",
    "suggestedAction": ""
  }
}
```

Cursor Prompt

Create src/services/aiService.ts.

```
Implement generateReflectiveResponse({
  message,
  category,
  emotion,
  riskLevel,
  verses
})
```

If riskLevel is CRISIS, return crisis response from safetyService.

Otherwise call an LLM API if API key exists.

If API key is missing or API fails, return a high-quality template response.

The response must include:

- pausePrompt
- answer
- verses
- forwardMirror.impulsivePath
- forwardMirror.reflectivePath
- forwardMirror.suggestedAction

Use the AatmaMirror constitution:

- reflection-first
- no blind validation
- no medical diagnosis
- Bhagavad Gita-inspired
- short, calm, practical

Output of Phase 6

- AI response generated
 - Fallback works even without API
 - Structured response ready for UI
-

Phase 7: Core Chat Flow

Goal

Connect UI + safety + RAG + AI.

Chat Flow

```
graph TD
    A[User selects category] --> B[User sends message]
    B --> C[detectEmotion()]
    C --> D[detectRiskLevel()]
    D --> E[If crisis → show CrisisAlert]
    E --> F[retrieveRelevantVerses()]
    F --> G[show PausePulse loading]
    G --> H[generateReflectiveResponse()]
    H --> I[render response cards]
    I --> J[ ]
```


decrease free query count

ChatScreen State

```
const [messages, setMessages] = useState<ChatMessage[]>([]);  
const [isReflecting, setIsReflecting] = useState(false);  
const [freeQueriesLeft, setFreeQueriesLeft] = useState(5);
```

Message Type

```
type ChatMessage = {  
  id: string;  
  role: "user" | "assistant";  
  text: string;  
  responseData?: ReflectiveAIResponse;  
  createdAt: string;  
};
```

Cursor Prompt

Update ChatScreen to implement full chat flow.

When user sends message:

1. Add user message
2. Detect emotion and risk
3. If crisis, show CrisisAlert response
4. Else retrieve verses
5. Show PausePulse loading for 3 seconds
6. Generate reflective response
7. Add assistant message with structured response cards
8. Reduce free query count

Render:

- EmotionBadge
- RiskBadge
- PausePulseCard
- VerseCard
- ForwardMirrorCard

Output of Phase 7

- Full user chat demo works
- Judges can test live

- Core product complete
-

Phase 8: Admin Content Generator

Goal

Generate social media content from mental wellness topics.

File

src/services/contentService.ts

Admin Inputs

- Topic
- Content type

Content Types

REEL

BLOG

CAPTION

IMAGE_PROMPT

Prompt

Generate mental wellness awareness content inspired by Bhagavad Gita teachings.

Topic: {topic}

Type: {type}

Rules:

- Keep it youth-friendly
- Do not sound preachy
- Encourage reflection
- Add practical takeaway
- Add hashtags if social content

AdminScreen Output

For Reel:

Hook:

Script:

CTA:

Caption:

Hashtags:

For Blog:

Title:

Intro:

3 Key Points:

Conclusion:

Cursor Prompt

Create src/services/contentService.ts and connect it to AdminScreen.

Admin can enter topic and select content type:

- Reel
- Blog
- Caption
- Image Prompt

Generate content using LLM if API key exists.

Fallback to template content if API fails.

Render output in a clean card with copy button.

Output of Phase 8

- Admin demo ready
- Content generation feature works
- Strong startup/business angle

Phase 9: Query Limit + Subscription Mock

Goal

Show monetization logic without real payment.

Free Limit

5 free queries

Behavior

If free queries left > 0:

- Allow chat

If free queries left = 0:

- Show subscription modal

Subscription Modal

Content:

You have used your 5 free reflections.
Upgrade to Premium for unlimited guidance.
₹99/month

Buttons:

- Upgrade Now
- Maybe Later

For demo, clicking Upgrade Now resets query count or marks premium.

Store

Use Zustand:

```
type AppStore = {  
  freeQueriesLeft: number;  
  isPremium: boolean;  
  decrementQuery: () => void;  
  upgradeToPremium: () => void;  
};
```

Cursor Prompt

Create Zustand store for app state.

Track:

- freeQueriesLeft default 5
- isPremium default false
- selectedCategory
- chatHistory

In ChatScreen:

- decrement free query after successful response
- if freeQueriesLeft is 0 and not premium, show subscription modal
- upgrade button sets isPremium true

Output of Phase 9

- Monetization story visible
 - Hackathon judges see business model
-

Phase 10: Testing + Demo Scenarios

Goal

Make sure the app works reliably.

Test Queries

Exam Stress

I am stressed about exams and feel like I will fail.

Expected:

- Emotion: stress
 - Risk: LOW
 - Verse: 2.47
 - Response: effort over results
-

Anger

I am angry at my friend and want to insult him.

Expected:

- Emotion: anger
 - Risk: MEDIUM
 - Verse: 2.63
 - Response: pause before reacting
-

Relationship

My girlfriend ignored me and I feel hurt.

Expected:

- Emotion: sadness/relationship
 - Risk: LOW
 - Response: reflection, communication, non-impulsive action
-

Crisis

I don't want to live anymore.

Expected:

- Risk: CRISIS
 - CrisisAlert shown
 - No normal spiritual advice
-

Admin

Topic: Exam stress

Type: Reel

Expected:

- Hook
- Script
- Caption
- Hashtags

Cursor Prompt

Add a DemoScreen with buttons for preset test scenarios:

- Exam Stress
- Anger
- Relationship Hurt
- Crisis
- Admin Reel Generation

Each button should navigate to the relevant screen and prefill/run the demo query.

Output of Phase 10

- Demo is reliable
 - No need to type during presentation
 - Backup scenarios ready
-

Phase 11: Deployment + Final Demo

Goal

Make the app presentable.

For Hackathon Demo

Use:

Expo Go

Steps:

`npx expo start`

Scan QR on phone.

Optional APK Build

`npm install -g eas-cli`

`eas login`

`eas build:configure`

eas build -p android --profile preview

Final Checklist

- App opens on phone
- Chat works
- AI response works
- Crisis demo works
- Admin content works
- UI looks polished
- No console errors
- Backup screenshots ready
- Backup screen recording ready

Cursor Prompt

Polish the app for final hackathon demo.

Add:

- loading states
- error handling
- empty states
- copy buttons
- clean spacing
- consistent dark theme
- demo mode buttons
- graceful fallback if AI API fails

Ensure app runs in Expo Go without errors.

Exact Build Order for Cursor

Use this order.

Prompt 1: Setup

Create the React Native Expo TypeScript app structure for AatmaMirror with navigation and placeholder screens.

Prompt 2: UI

Build polished mobile UI for Onboarding, Category, Chat, Admin, and Demo screens using dark gradient theme.

Prompt 3: Dataset

Create local Bhagavad Gita dataset with 30 verse objects for student mental wellness categories.

Prompt 4: Safety

Implement emotion detection, risk detection, and crisis response service.

Prompt 5: RAG

Implement local RAG retrieval using tag/category scoring from the Gita dataset.

Prompt 6: AI

Implement AI response service with LLM API call and fallback template response.

Prompt 7: Chat

Connect ChatScreen to safety, RAG, and AI services. Render structured response cards.

Prompt 8: Admin

Build Admin content generator for reels, blogs, captions, and image prompts.

Prompt 9: Monetization Mock

Add 5 free query limit and subscription modal using Zustand.

Prompt 10: Demo Polish

Add demo scenarios, loading states, error handling, and final UI polish.

Final 3-Day Execution Schedule

Day 1: Mobile Foundation

Phase 0: Scope lock

Phase 1: Project setup
Phase 2: UI screens
Phase 3: Gita dataset

Day 2: AI Core

Phase 4: Emotion + risk detection
Phase 5: RAG retrieval
Phase 6: AI response engine
Phase 7: Core chat flow

Day 3: Business + Demo

Phase 8: Admin content generator
Phase 9: Query limit/subscription mock
Phase 10: Testing/demo scenarios
Phase 11: Expo deployment/demo prep

Final MVP Demo Flow

Show this to judges:

1. Student selects category

Exam Stress

2. Student asks

I am stressed about exams and feel like I will fail.

3. App shows

Emotion: Stress
Risk: Low
PausePulse: Take a moment before reacting
Verse: Bhagavad Gita 2.47
Guidance: Focus on effort, not fear of result
ForwardMirror: Panic path vs reflective path

4. Crisis demo

I don't want to live anymore.

App shows:

Crisis Escalator activated

Immediate human support message

5. Admin demo

Topic: Exam Stress

Type: Reel

App generates:

Hook + Script + Caption + Hashtags

Final Engineering Recommendation

For 3 days, do **not** over-engineer.

Build:

React Native Expo

+ Local Gita JSON

+ Rule-based emotion/risk detection

+ Local RAG scoring

+ LLM response generation

+ Crisis fallback

+ Admin content generator

After hackathon, upgrade to:

Supabase PostgreSQL + pgvector

Real auth

Real subscriptions

Voice input/output

Counsellor dashboard

Multi-scripture support